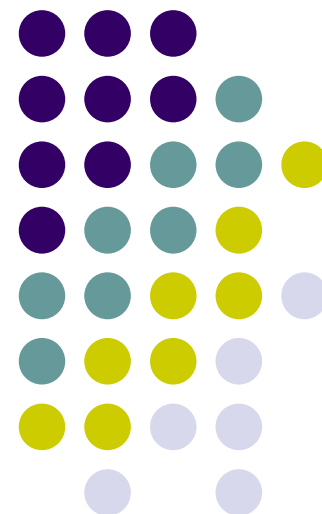
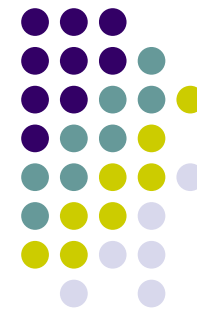


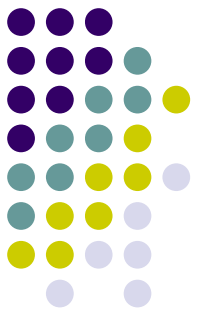
Ch3 进程编程



本章内容

- 进程和进程环境
- 进程控制
- 进程关系
- 守护进程**Daemon**

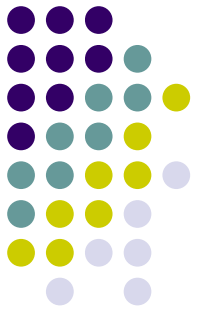




Part 1 进程和进程环境

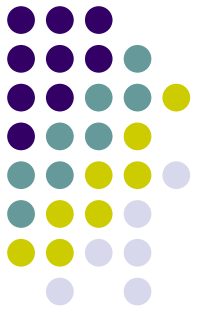
1. 进程

- Process: an **address space** with one or more **threads** executing within that address space, and the required **system resources** for those threads.



2.进程启动

- System call “fork”
Process resources
 - struct task_struct
 - System space stack
 - ...
- System call “exec”
- The entry of C programs



3.进程终止

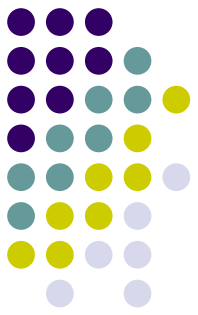
- 进程终止的五种方式

- Normal termination

- Return from “main” function
- Call “exit” function
- Call “_exit” function 不处理缓冲区

- Abnormal termination 异常结束

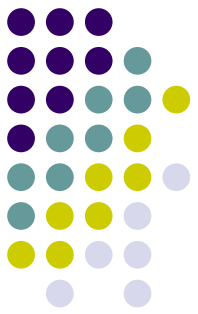
- Call “abort” function
- Terminated by a signal 信号



4.进程分类

●Foreground process前台进程

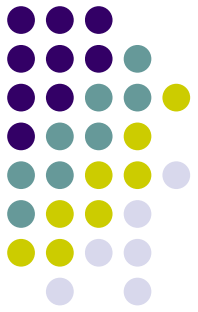
- 要求用户启动它们或与它们交互的进程称为前台进程。
- 前台进程不结束，终端就不会出现系统提示符，直到进程终止。
- 缺省情况下，程序和命令作为前台进程运行。



4.进程分类

● Background process 后台进程

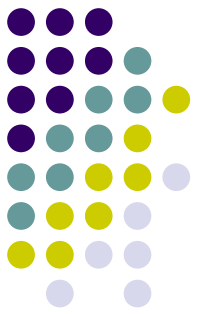
- 独立于用户运行的进程称为后台进程。
- 用户在输入命令行后加上 “&” 字符然后按 <Enter> 键就启动了后台进程。
- Shell 不等待命令终止，就立即出现系统提示符，让该命令进程在后台运行，用户可以继续执行新的命令。



4.进程分类

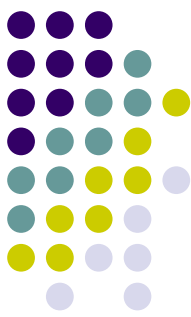
● Daemon 守护进程

- 总是运行在后台的系统进程。
- 守护程序通常在系统启动时启动，并且它们一直运行到系统停止。
- 守护进程常常用于向用户提供各种类型的服务和执行系统管理任务。
- 守护程序进程由 **root** 用户或 **root shell** 启动，并只能由 **root** 用户停止。



Part 2 进程控制

- 进程标识符: `getpid, getppid`
- 进程创建: `fork, vfork, system, exec`
- 进程终止: `exit, _exit`
- 父子进程关系: 简单同步 `wait, waitpid`



2.1 进程标识符

```
#include <sys/types.h>
#include <unistd.h>
```

```
pid_t getpid(void);
```

返回：调用进程的进程 ID

```
pid_t getppid(void);
```

返回：调用进程的父进程 ID

```
uid_t getuid(void);
```

返回：调用进程的实际用户 ID

```
uid_t geteuid(void);
```

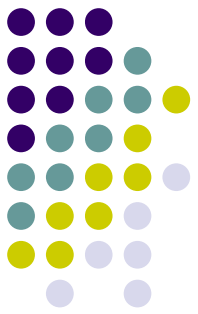
返回：调用进程的有效用户 ID

```
gid_t getgid(void);
```

返回：调用进程的实际组 ID

```
gid_t getegid(void);
```

返回：调用进程的有效组 ID



2.2 进程创建

1.fork

- fork: create a child process

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

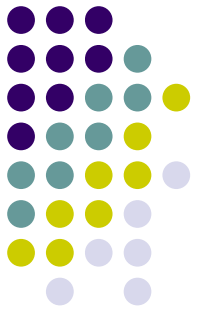
```
pid_t fork(void);
```

- returned value:

pid of child (in the current (parent) process),

0 (in child process),

-1 (failure)



2.2 进程创建

1.fork创建子进程代码结构

.....

```
pid = fork();  
if (pid<0) {perror("fork()");exit(1);}  
else if (pid==0) { child process }  
else { parent process }
```

文件共享



- 所有由父进程打开的描述符都被复制到子进程中。父、子进程每个相同的打开描述符共享一个文件表项

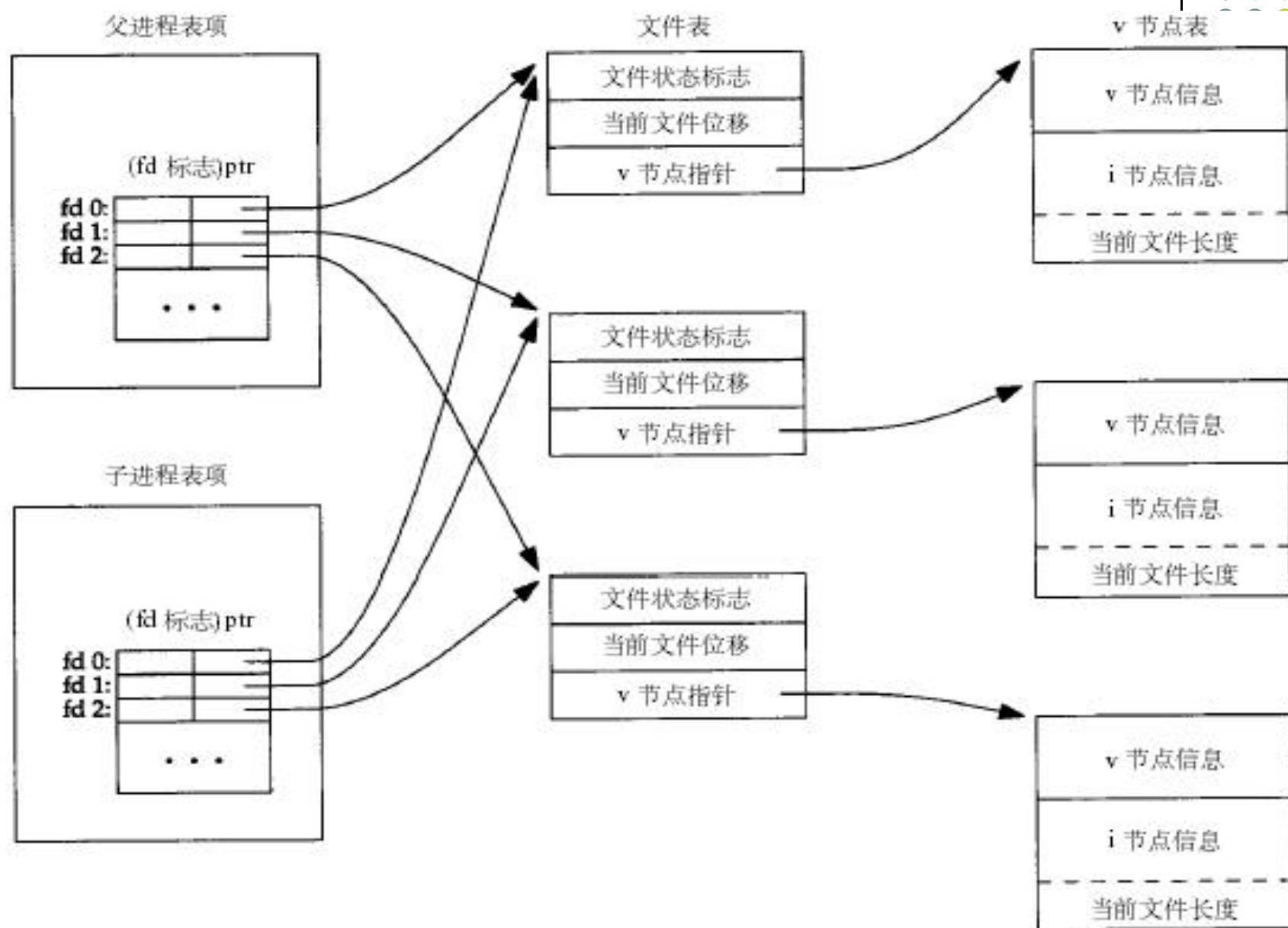
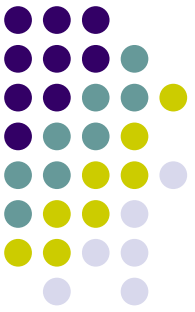
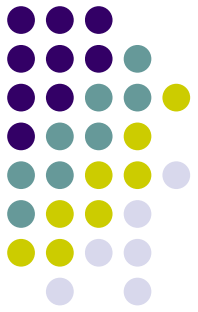


图8-1 fork之后父、子进程之间对打开文件的共享



Example3-1

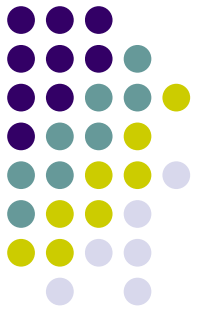
```
int main(int argc, char *argv[])
{
    pid_t pid;
    int fd;
    int i=1;
    int status;
    char *ch1="hello";
    char *ch2="world";
    char *ch3="IN";
    if((fd=open("test.txt", O_RDWR|O_CREAT, 0644))==-1)
    { perror("parent open"); exit(EXIT_FAILURE); }
    if(write(fd, ch1, strlen(ch1))==-1)
    { perror("parent write"); exit(EXIT_FAILURE); }
```



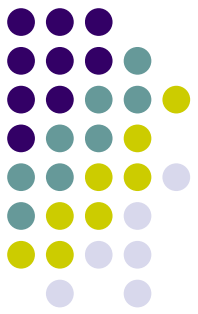
Example3-1

```
if((pid=fork())==-1)
{ perror("fork"); exit(EXIT_FAILURE); }
else
if(pid==0)
{ i=2;
printf("in child\n"); //打印 i 值, 以示与父亲进程区别
printf("i=%d\n",i);
if(write(fd,ch2,strlen(ch2))==-1) //写文件 test.txt, 与父进程共享
perror("child write");
return 0; }
else
{ sleep(1); //等待子进程先执行
printf("in parent\n");
printf("i=%d\n",i); //打印 i 值, 以示与子进程区别
if(write(fd,ch3,strlen(ch3))==-1) //写操作, 结果添加到文件后
perror("parent,write");
wait(&status); //等待子进程结束
return 0; }
}
```


Example3-2

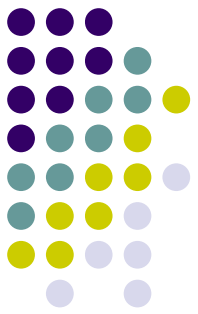


```
main()
{ int i,j,mark;
  for (i=LEFT;i<=RIGHT;i++)
  { mark=1;
    for (j=2;j<i/2;j++)
    { if (i%j==0)
      { mark=0;
        break;
      }
    }
    if (mark)
      printf("%d is a primer!\n",i);
  }
}
```



Example3-2——单个进程

```
#define LEFT 30000001
#define RIGHT 30000200
main()
{ int i,j,mark;
  for (i=LEFT;i<=RIGHT;i++)
  { mark=1;
    for (j=2;j<i/2;j++)
    { if (i%j==0)
      { mark=0;
        break;
      }
    }
    if (mark)
      printf("%d is a primer!\n",i);
  }
}
```



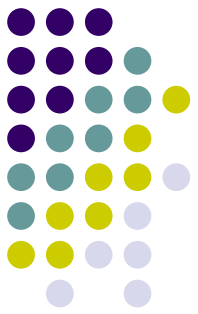
Example3-2——多个进程

```
main()
{ int i,j,mark;
  pid_t pid;
  for (i=LEFT;i<=RIGHT;i++)
  { mark=1;
    pid = fork();
    if (pid==0)
    { for (j=2;j<i/2;j++)
      { if (i%j==0)      { mark=0;          break;          }
      }
      if (mark) printf("%d is a primer!\n",i);
    }
  }
  exit(0);
}
```

有无问题??

又错了。

执行系统



fork应用场合

- 进程复制自己，使父子进程同一时刻执行不同的代码——网络服务
- 进程要执行另一个不同的程序：fork-exec——shell

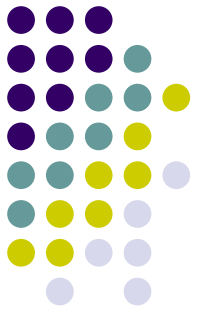
Question: 效率问题？父子进程各自占一段逻辑地址空间，fork之后立即exec，地址空间浪费。

“写—复制” COW
vfork

fork → copy on write

2. vfork

→ 与 fork 的区别是 子进程共用父进程的映像

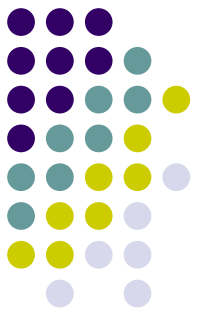


- vfork

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t vfork(void);
```

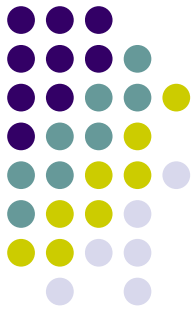


2.vfork

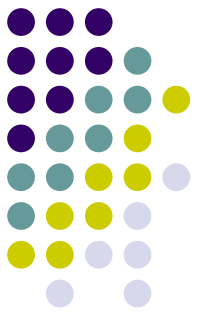
- 功能：类似fork，创建一个新进程，效率高。
- 与fork区别：
 - (1) vfork创建的进程与父进程共用地址空间
 - (2) vfork创建子进程后，阻塞父进程，直到子进程调用exec或exit，内核才唤醒父进程。

Example 3-3-1

fork 创建子进程

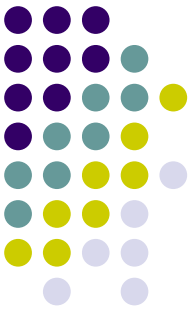


```
int global=5;
main()
{int pid; char *string="I am father: "; int local=10;
printf("before fork\n");
if((pid=fork())<0) {perror("fork failed");exit(0);}
if(pid==0)
{string="I am child.";
global++;
printf("%s global=%d, local=%d\n",string,global,local);
exit(0); }
else
{local++;
printf("%s my pid is %d \n" "global=%d\n local=%d\n",
string,getpid(),global,local);
exit(0);
}}
```



Example3-3-2

```
int global=5;
main()
{int pid; char *string="I am father: "; int local=10;
 printf("before vfork\n");
 if((pid=vfork())<0) {perror("vfork failed");exit(0);}
 //共用进程空间，子进程先执行
 if(pid==0)
 {string="I am child.";
  global++;
  printf("%s global=%d, local=%d\n",string,global,local);
  exit(0); }
 else
 {local++;
  printf("%s my pid is %d \n" "global=%d\n local=%d\n",
  string,getpid(),global,local);
  exit(0);
 }}
```

相关函数

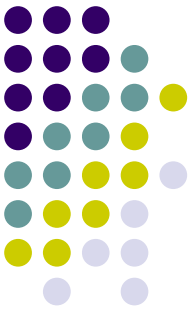
- sleep函数

函数原型:

```
#include <unistd.h>
```

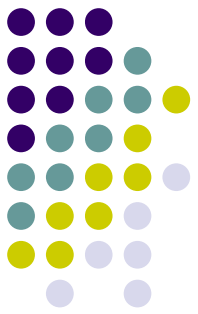
```
unsigned int sleep(unsigned int seconds);
```

seconds: 暂停时间（秒）



3.exec 系列函数

- Replace the current process image with a new process image.
- 执行新程序的进程保持原进程的一系列特征：
 - pid, ppid, uid, gid, working directory, root directory ...



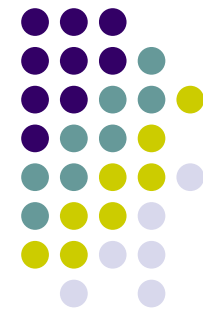
函数原型

```
#include <unistd.h>
extern char **environ;
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execlp(const char *file, const char *arg, ...,
char * const envp[]);
int execv(const char *path, char * const argv[]);
int execvp(const char *file, char * const argv[]);
int execve(const char *filename, char * const argv[],
char * const envp[]);
```

l → hse
子过程

√ → 父过程

exec 系列函数



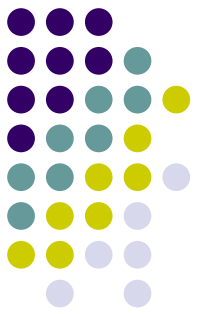
1. 两大类:

execl开头: 参数以列表形式`arg0,arg1...NULL`结束。

execv开头: 参数以指向字符串数组`argv[]`指针形式,
`arg[0]`必须为程序名。

2. 含有字母**p**的函数可以使用相对路径, 根据环境变量
PATH查找文件; 其他函数必须使用绝对路径;

3. 含有字母**e**的函数, 需要通过指向**envp[]**数组的指针指明新的环境变量, 其他函数使用当前环境变量。



exec 举例

例如：用exec函数使新进程执行 “/bin/ps” 程序。

```
#include<unistd.h>
```

```
const char*ps_argv[]={“ps”,“-af”, NULL};
```

```
const char *ps_envp[] = {“PATH=/bin:/usr/bin”,  
    “TERM=console”, NULL};
```

六种情况：

```
execl(“/bin/ps”,“ps”,“-af”,NULL);
```

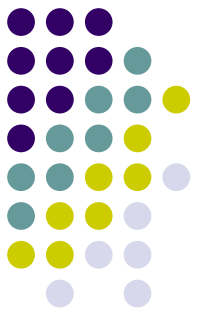
```
execvp(“ps”,ps_argv);
```

```
execve(“/bin/ps”,ps_argv,ps_envp);
```

```
execvp(“ps”,ps_argv);
```

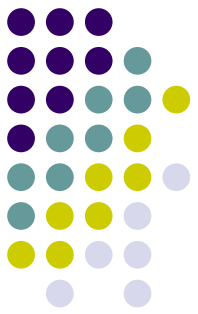
```
execvp(“ps”,ps_argv);
```

```
execve(“/bin/ps”,ps_argv,ps_envp);
```



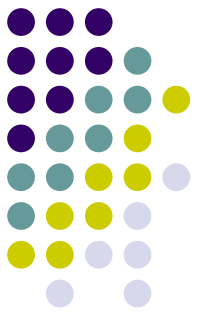
fork和exec一起使用

- Two ways of using fork
 - The parent process duplicates itself, and then two different pieces of codes are executed in parent process and child process. 父子进程各自执行不同的代码
 - A process want to execute another program. 进程要执行另一个程序



Example: a simple shell

```
printf("%% ");    /* print prompt */
while (fgets(buf, MAXLINE, stdin) != NULL) {
    buf[strlen(buf) - 1] = 0;    /* replace newline with null */
    if ( (pid = fork()) < 0 )
        err_sys("fork error");
    else if ( pid == 0 ) {          /* child */
        {
            execlp(buf, buf, (char *) 0);
            fprintf(stderr, "couldn't execute: %s", buf);
            exit(127);
        }
    }
    if ( (pid = waitpid(pid, &status, 0)) < 0 ) /* parent */
        err_sys("waitpid error");
    printf("%% ");
}
```



2.3 进程终止

1.exit函数

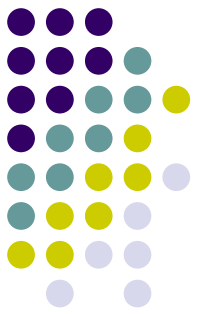
函数原型:

```
#include <stdlib.h>
```

```
void exit(int status);
```

status:进程状态

功能：正常终止目前进程的执行，把参数**status**返回给父进程，进程所有的缓冲区数据会自动写回并关闭未关闭的文件。



2._exit函数

函数原型:

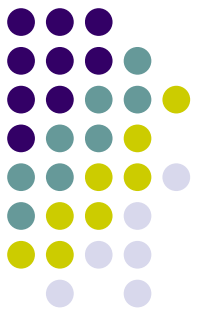
不处理缓冲

`#include <stdlib.h>`

`void ——exit(int status);`

status:进程状态

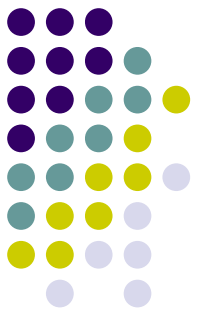
功能：立刻终止目前进程的执行，把参数**status**返回给父进程，并关闭未关闭的文件。不处理标准I/O缓冲区。



2.4 父子进程关系

1. 两种进程概念

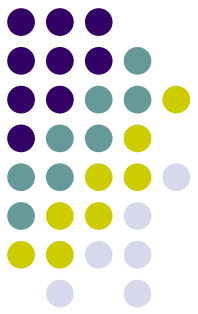
- 父进程在子进程前终止——孤儿进程
 - Orphan process——init
- 子进程在父进程前终止——可能成为僵尸进程
 - SIGCHLD signal 忽略 SIGCHLD 信号 未响应信号
 - Handled by wait/waitpid in parent 父进程中用 wait/waitpid 处理
 - Not handled by wait/waitpid in parent -> zombie 父进程没有用 wait/waitpid 处理 -> 僵尸进程



2. 孤儿进程 (Orphan)

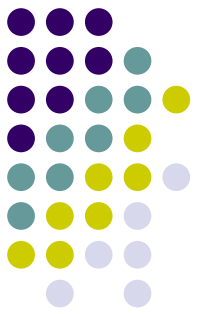
孤儿进程：父进程终止，子进程未终止，子进程成为孤儿进程。

Example 3-4 process_orphan.c



```
int main(int argc, const char *argv[])
{
    pid_t pid;
    pid = fork();
    if(pid < 0){
        perror("fork error");
        return -1;
    }
    else if(pid == 0){
        printf("The child process, id = %d parent id = %d\n",getpid(),
getppid());
        sleep(1);
        printf("The child process, id = %d parent id = %d\n",getpid(),
getppid());
        while(1);
    }
    else{ sleep(1);    printf("The parent process, id = %d\n",
getpid());    }
}
```

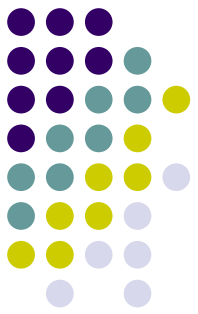
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
```



3. 僵尸进程（**Zombie**）

- 僵尸进程：已终止运行，但尚未被清除的进程。
- 子进程运行结束后（正常或异常），它并没有马上从系统的进程分配表中被删掉，而是进入僵死状态（**Zombie**），一直等到父进程来回收它的结束状态信息。

Example 3-4 process_zombie.c

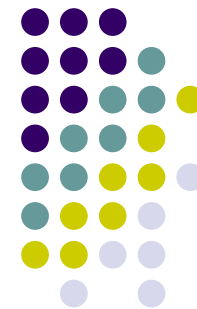


```
int main(int argc, const char *argv[])
{   pid_t pid;
    pid = fork();
    if(pid < 0){
        perror("fork error");
        return -1;
    }
    else if(pid == 0){
        printf("The child process, id = %d parent id = %d\n",
               getpid(), getppid());
    }
    else{
        printf("The parent process, id = %d\n", getpid());
        while(1);
    }
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
```

@defunct>

子进程之状态



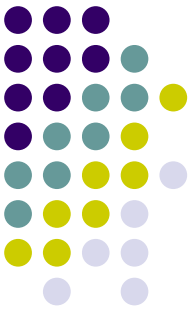
4. 僵尸进程解决方法

- 僵尸进程的proc结构一直存在直到父进程正常结束或系统重启。——如何消除僵尸进程？
- 方法一：wait/waitpid阻塞父进程，子进程先终止
- 方法二：父进程不阻塞，两次fork
- 方法三：使用signal信号处理

方法一：wait example



- 例3.5设计一个程序，要求复制进程，子进程显示自己的进程号后暂停一段时间，父进程等待子进程正常结束，打印显示等待的进程号和等待的进程退出状态。



Example 3-5-1wait.c

```
main()
{ int i,j,mark; pid_t pid;
  for (i=LEFT;i<=RIGHT;i++)
  { mark=1;
    pid = fork();
    if (pid==0)
    {
      for (j=2;j<i/2;j++)
      if (i%j==0)      { mark=0;      break;      }
      if (mark)      printf("%d is a primer!\n",i);
      sleep(100);
      exit(0);
    } }
  for (i=LEFT;i<=RIGHT;i++)
    wait(NULL);
    exit(0);
}
```

反进程等待子进程。

系统中的第一个终止的子进程。

wait & waitpid functions

回收指定的子进程。

- Wait for process termination
- Prototype

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

waitpid的第一个参数pid的意义：

pid > 0: 等待进程id为pid的子进程。

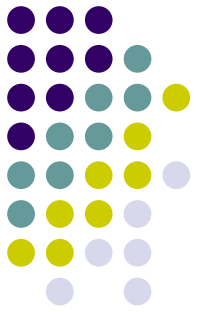
pid == 0: 等待与自己同组的任意子进程。

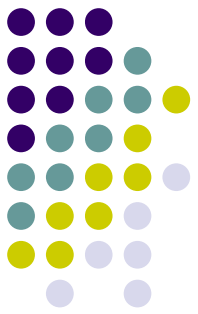
pid == -1: 等待任意一个子进程

pid < -1: 等待进程组号为-pid的任意子进程。

因此，wait(&stat)等价于waitpid(-1, &stat, 0)

waitpid例子

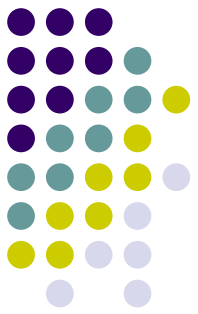




Example 3-5-2waitpid.c

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<stdlib.h>
int main ()
{   pid_t pid,wpid;           int status,i;
    pid=fork();
    if(pid==0)
    {   printf("这是子进程,进程号(pid)是:%d\n",getpid());
        sleep(5);           exit(6); }
    else
    {   printf("这是父进程,正在等待子进程.....\n");
        wpid=waitpid(pid,&status,0);
        i=WEXITSTATUS(status);
        printf("等待的进程的进程号(pid)是:%d ,结束状态:%d\n",wpid,i);
    }
}
```

父进程阻塞。



方法二：两次fork Example3-6.c

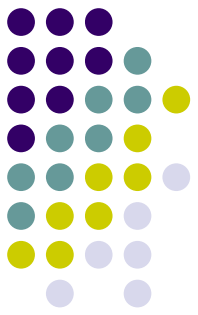
```
int main(void)
{pid_t pid;
 if((pid=fork())<0) perror("fork error");
 else if(pid==0) /*first child*/
 { if((pid=fork())<0) perror("fork error");
  else if(pid>0) exit(0); /*parent of second child=first child*/
  sleep(10); /*second child*/
  printf("second child,parent pid=%d\n",getppid());
  exit(0);
 }
 if (waitpid(pid,NULL,0)!=pid) /*wait for first child,parent(the original
 process)*/
  perror("waitpid error");
 printf("parent exit");
 exit(0);
}
```

Diagram illustrating the process flow:

- Process A (Parent) forks to create Process B (First child).
- Process B forks to create Process C (Second child).
- Process C sleeps for 10 seconds.
- Process C prints the parent PID (which is Process B's PID).
- Process C exits.
- Process B waits for Process C to finish.
- Process B prints the parent PID (which is Process A's PID).
- Process B exits.
- Process A waits for Process B to finish.
- Process A prints the parent PID (which is its own PID).
- Process A exits.

Handwritten notes in Chinese:

- 父 (Father) - next to Process A
- 子 (Son) - next to Process B
- 孙子 (Grandson) - next to Process C
- 结束 (End) - next to the end of the code
- 回收进程 (Recycle process) - next to the waitpid call
- 成为父进程 (Become parent process) - next to Process C's parent PID print



方法三：singnal函数处理

- Singnal函数处理：

(1) 设置信号处理函数

`signal(SIGCHLD, fun)`

(2) 忽略子进程终止信号

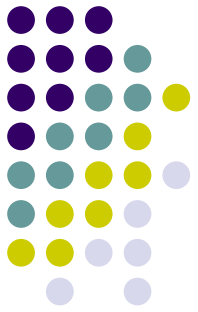
`signal(SIGCHLD, SIG_IGN)`

内核回收

子父进程回收

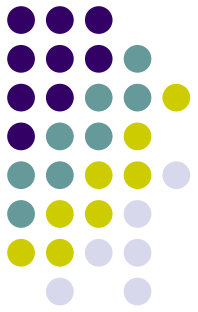
函数

fun 函数名
回收



掌握的知识点

- fork和vfork的异同
- exec系列函数的使用形式
- exit和_exit函数区别
- 僵尸进程和孤儿进程概念
- 僵尸进程处理方法



Part 4 守护进程Daemon

- 登录方式
- 进程组和会话期
- 守护进程

4.1 登录方式(1)

- 终端登录

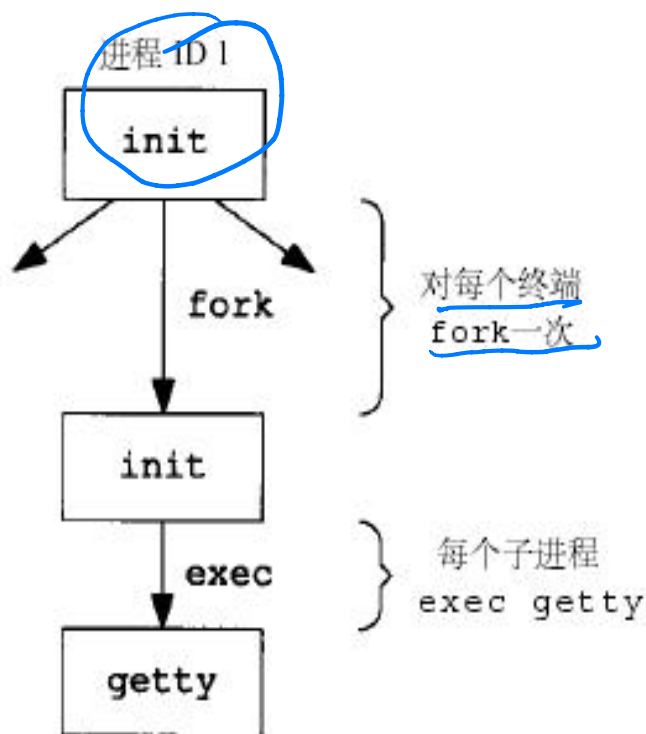


图9-1 init生成进程使终端可用于登录

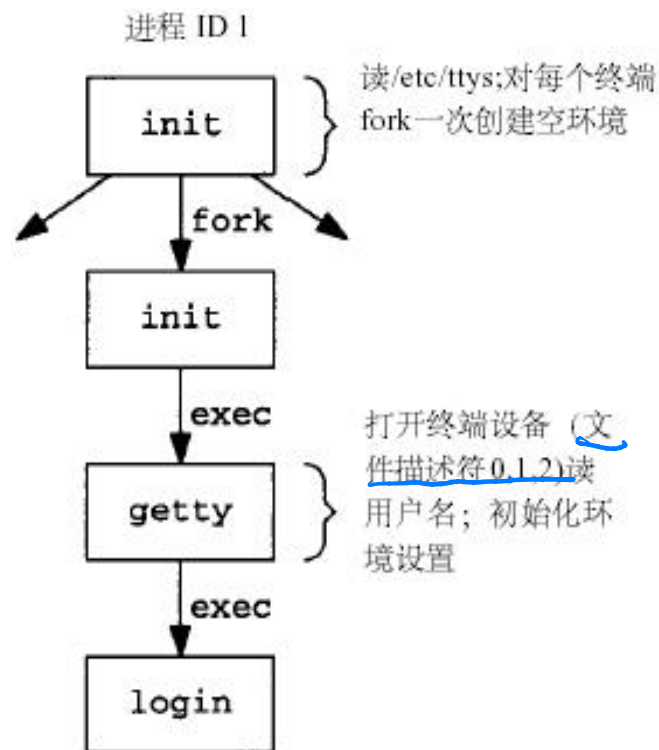
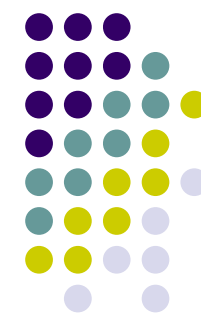


图9-2 login刚被调用后各进程的状态



4.1 登录方式(2)

- 网络登录

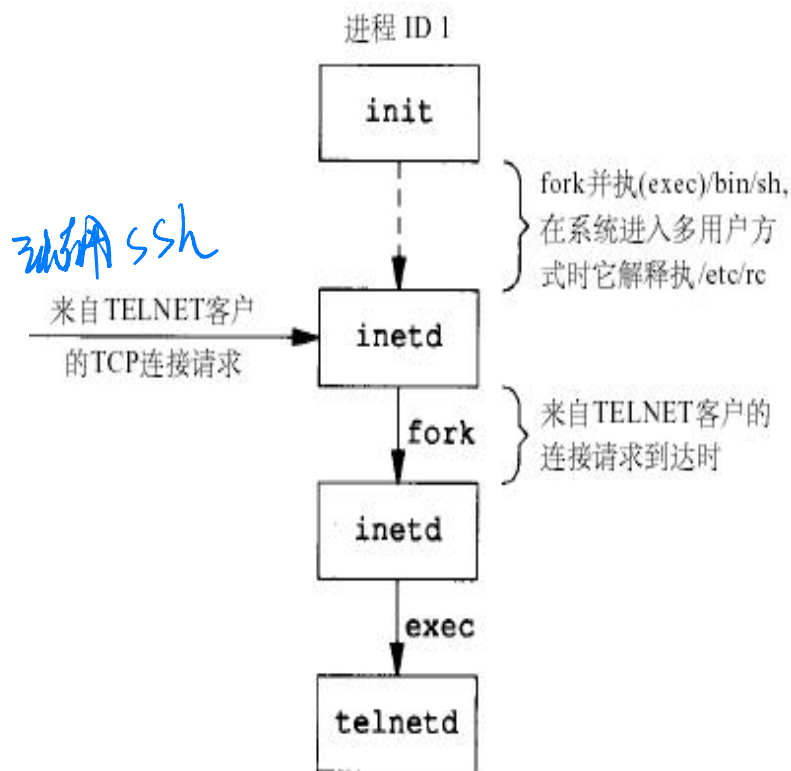


图9-4 执行TELNET服务进程中涉及的进程序列

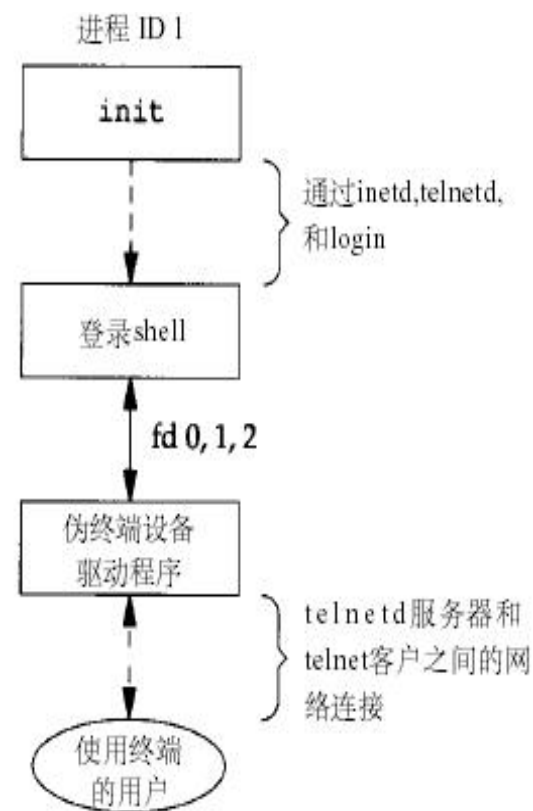
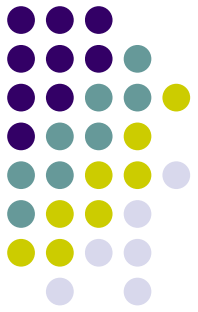


图9-5 为网络登录设置了 fd 0, 1, 2 后的进程安排



4.2 进程组和会话期

- 进程组

- The set of one or more process(es).
- `getpgrp/setpgid` functions

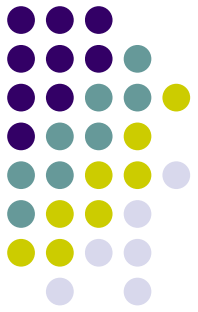
- 进程组长

- 会话组 Session

- The set of one or more process group(s).
- `getsid/setsid` function
- 会话组长

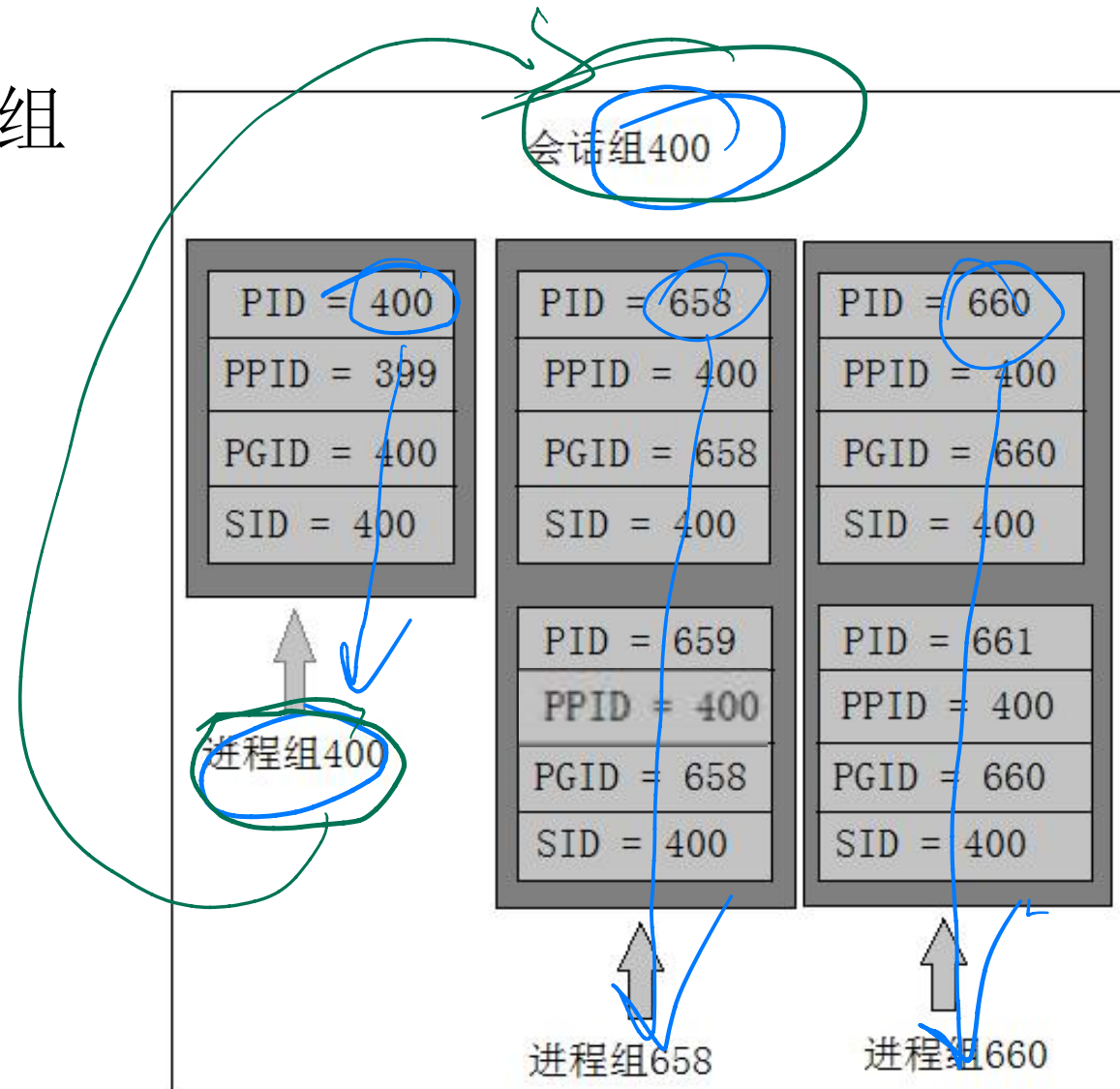
有一个组长进程

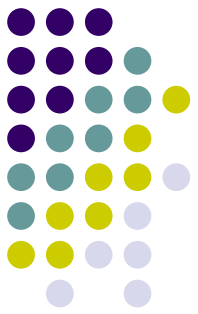
—— 所有进程都是它的子进程



4.2 进程组和会话期

- 进程组和会话组





4.2 进程组和会话期

- `getpgrp/setpgid` 函数

```
#include <unistd.h>
```

```
pid_t getpgid(pid_t pid);
```

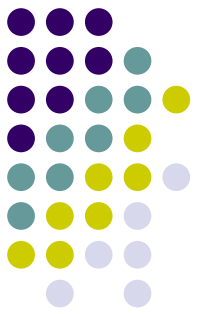
```
pid_t getpgrp(void);           /* POSIX.1 version */
```

```
pid_t getpgrp(pid_t pid);      /* BSD version */
```

`getpgid()` 函数返回进程号为参数 `pid` 的进程所属的进程组 ID，
如果参数 `pid` 为 0，则返回调用进程所属的进程组 ID

`getpgrp()` 函数用于返回调用进程所属的进程组 ID

`getpgrp()` 函数则返回进程号为参数 `pid` 的进程所属的进程组 ID



4.2 进程组和会话期

- `getpgrp/setpgid` 函数

```
#include <unistd.h>
```

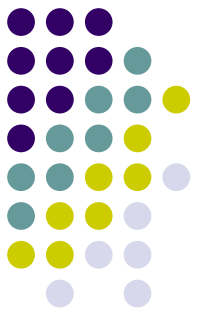
```
int setpgid(pid_t pid, pid_t pgid);
```

```
int setpgrp(void); /* System V version */
```

```
int setpgrp(pid_t pid, pid_t pgid); /* BSD version */
```

`setpgid()` 函数将进程号为参数 `pid` 的进程所属的进程组 ID 修改为参数 `pgid` 所表示的值。

`setpgrp()` 函数用于修改调用进程所属进程组的 ID 为调用进程的 ID（调用进程成为组长）



4.2 进程组和会话期

- getsid/setsid函数

```
#include <unistd.h>
```

```
pid_t getsid(pid_t pid);
```

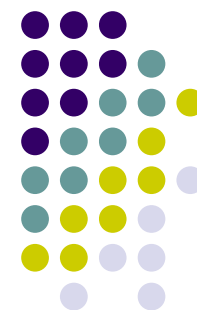
`getsid()`函数用于获得进程号为参数`pid`的进程所属的会话组的组ID。如果参数`pid`为0，则返回调用进程所属的会话组ID。

```
#include <unistd.h>
```

```
pid_t setsid(void);
```

`setsid()`函数会创建一个新会话，其前提是调用进程不能是进程组组长。

该函数所会话的组

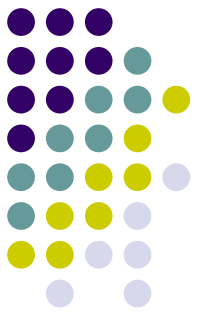


4.2 进程组和会话期

● setsid函数

所需头文件	<code>#include<sys/types.h></code> <code>#include<unistd.h></code>
函数功能	设置新的组进程号
函数原型	<code>pid_t setsid(void);</code>
函数传入值	无
函数返回值	执行成功则返回进程组号(gid)，失败则返回-1，失败原因存于errno中。
相关函数	setregid, setegid, getsid, setregid
备注	如果调用此函数的进程不是一个进程组的组长，则此函数创建一个新对话期，结果为：(1) 此进程变成该新对话期的对话期首进程。此进程是该新对话期中的唯一进程。(2) 此进程成为一个新进程组的组长进程。新进程组号是此调用进程的进程号。(3) 此进程没有控制终端。如果在调用 setsid 之前此进程有一个控制终端，那么这种联系也被解除。 如果此调用进程已经是一个进程组的组长，则此函数返回出错。为了保证不处于这种情况，通常先调用 fork，然后使其父进程终止，而子进程则继续。

取消原来
的控制终端



4.2 进程组和会话期

整个终端是一个 session

```
proc1 | proc2 &  
proc3 | proc4 | proc5
```

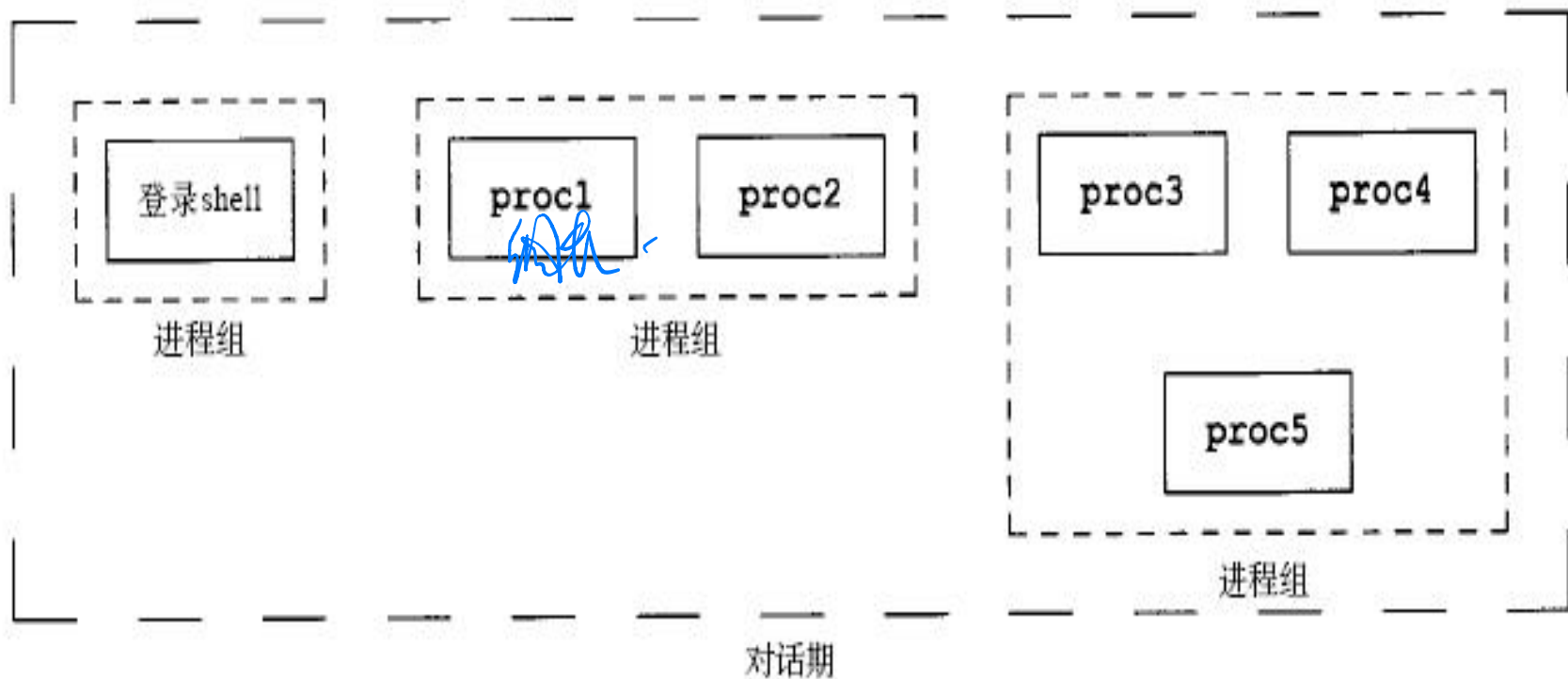
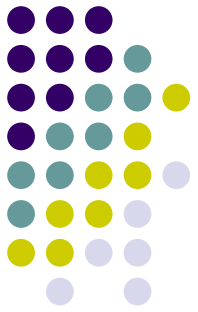


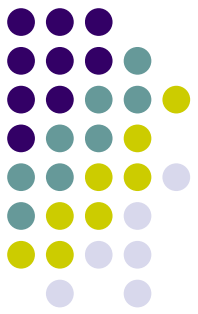
图9-6 进程组和会话期中的进程安排

会话期 = 一个终端的一个 session



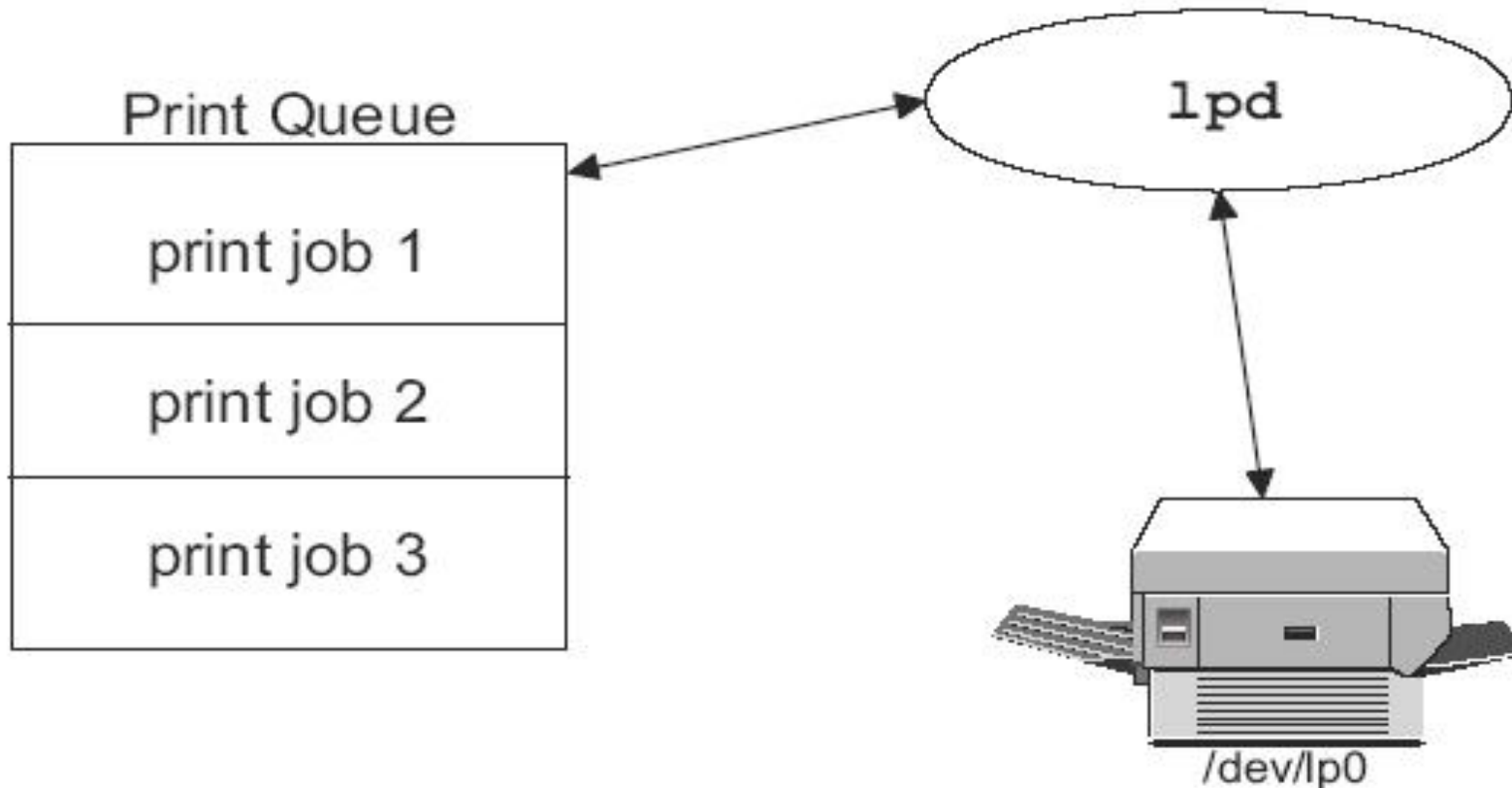
4.3 守护进程Daemon

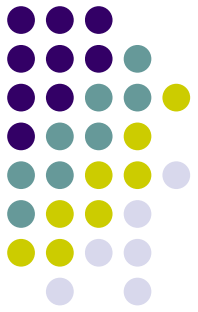
- What is a daemon?
- daemon进程的实现方法
 - 编程规则
 - 出错记录



4.3.1 What is a daemon?

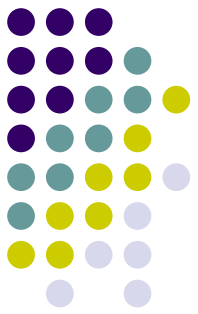
- The word "Daemon" refers to a never-ending process, usually a system process that controls a system resource such as the printer queue or performs a network service





What is a daemon? (cont'd)

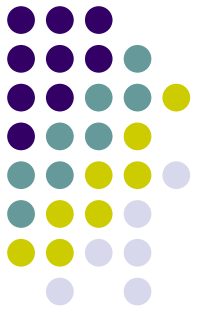
- daemon
 - 精灵进程或守护进程
 - 后台执行, 没有控制终端或登录 **Shell** 的进程



ps -aux 命令查看

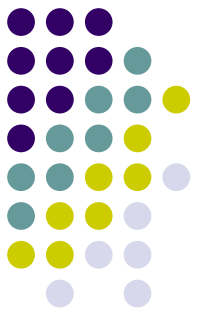
- init:进程1，启动系统服务
- Keventd: 为内核中运行的函数提供进程上下文
- Kswapd: 页面调出守护进程
- bdfush, kupdated: 调整缓存中的数据写到磁盘
- portmap: 将RPC程序号映射为端口号
- inetd(xinetd): 侦听网络接口，获取网络服务进程请求

注意：大多数守护进程都以超级用户（用户ID为0）特权运行。没有一个守护进程具有控制终端，其终端名设置为问号（?）。



ps axj命令查看

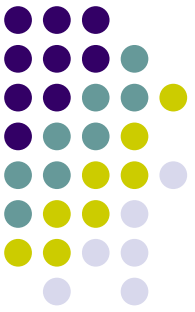
- daemon特征: *session id 进程 id 父子进程 pre*
 - sid, pid, pgid相同, 均为pid
 - ppid为1 *父进程为1号进程*
 - tty为?
不在终端



4.3.2 daemon进程实现规则

- 编程规则

- 首先调用fork，然后使父进程exit
- 调用setsid创建一个新的会话期 `setsid()`
- 将当前工作目录更改为特定目录chdir(./) *设置工作目录*
- 进程的umask设为0 `umask(0)`
- 关闭不需要的文件描述符 `close()`



Example

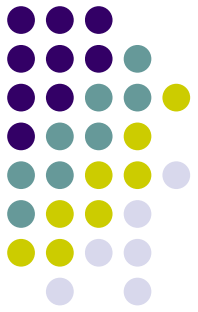
```
int daemon_init(void) {
    pid_t pid;

    if ( (pid = fork()) < 0)
        return(-1);
    else if (pid != 0)
        exit(0); /* parent goes bye-bye */

    /* child continues */
    setsid(); /* become session leader */
    chdir("/tmp"); /* change working directory */
    umask(0); /* clear our file mode creation mask */
    for(i=0;i<MAXFILE;i++) /* close file */
        close(i);
    return(0);
}
```

Handwritten notes in blue ink:

- Next to `fork()`: } ⇒ pp + 1/2
- Next to `setsid()`: 脱离进程, pid, pgid, 进程不挂
- Next to `chdir("/tmp")`: 设置工作目录
- Next to `umask(0)`: 设置掩码



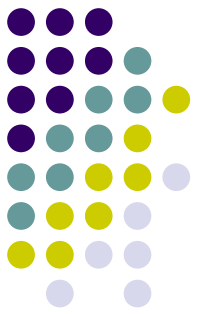
编写守护进程的要点

(1) 创建子进程，终止父进程

```
pid=fork();  
if(pid>0)  
{exit(0);} /*终止父进程*/
```

(2) 在子进程中创建新会话

- **setsid**函数用于创建一个新的会话，并担任该会话组的组长，其作用：
 - ① 让进程摆脱原会话的控制；
 - ② 让进程摆脱原进程组的控制；
 - ③ 让进程摆脱原控制终端的控制。
- 而**setsid**函数能够使进程完全独立出来，从而脱离所有其他进程的控制。



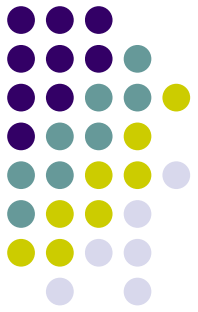
编写守护进程的要点

(3) 改变工作目录

- 改变工作目录的常见函数是`chdir`。

(4) 重设文件创建掩码

- 文件创建掩码是指屏蔽掉文件创建时的对应位。
- 把文件创建掩码设置为0，可以大大增强该守护进程的灵活性。
- 设置文件创建掩码的函数是`umask`。



编写守护进程的要点

(5) 关闭文件描述符

- 通常按如下方式关闭文件描述符：

```
for(i=0;i<NOFILE;i++)
```

```
close(i);
```

- 或者也可以用如下方式：

```
int num;
```

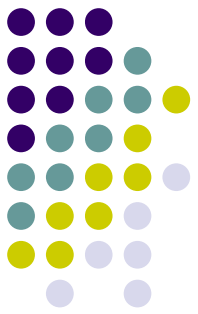
```
num = getdtablesize();
```

```
/*获取当前进程文件描述符表大小*/
```

```
for (i = 0; i < num; i++) {
```

```
close (i) ;
```

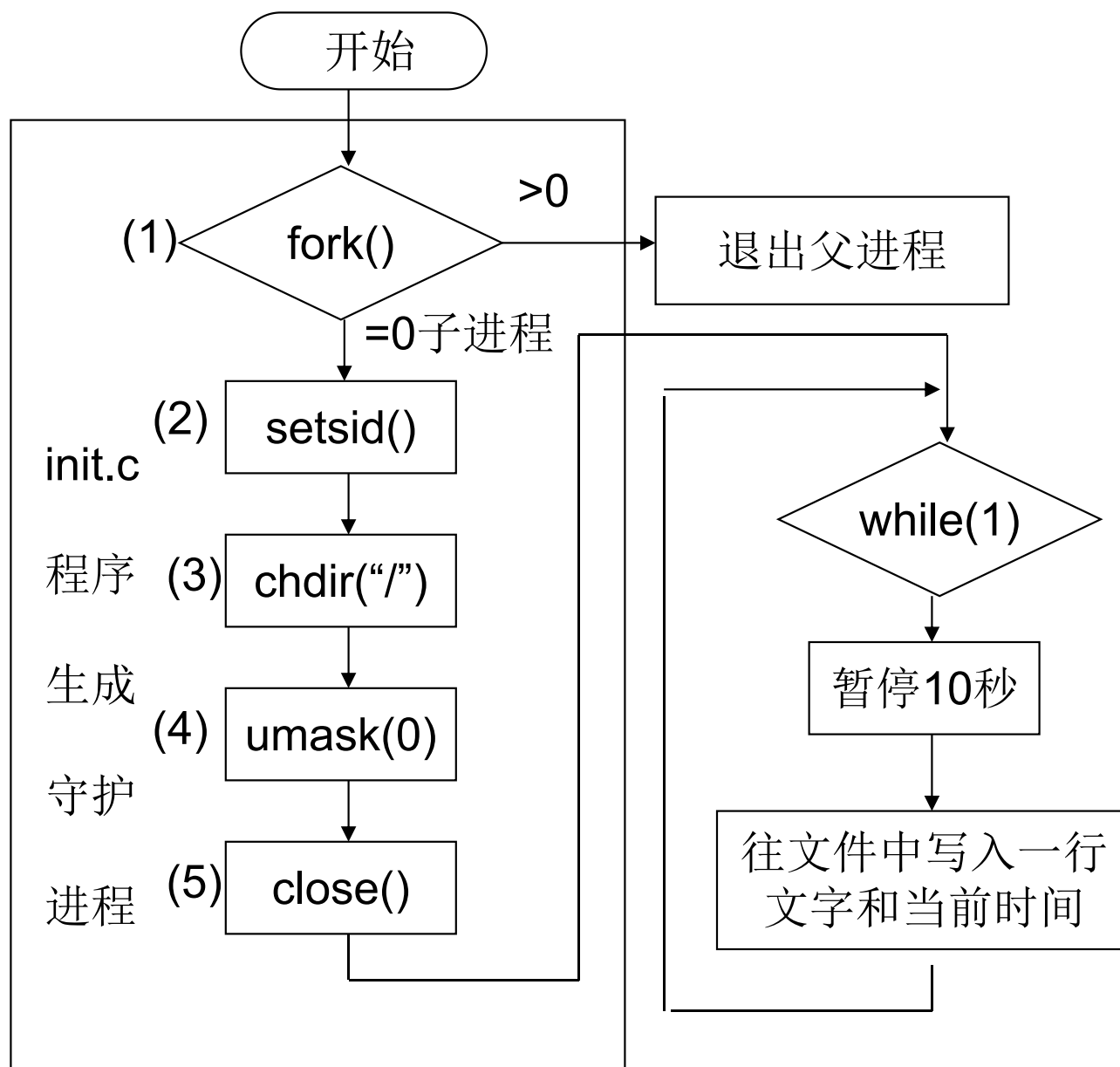
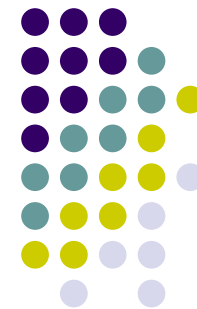
```
}
```



4.3.3 守护进程编写

- 例3.7 设计两个程序，主程序和初始化程序。要求主程序每隔10秒向/tmp目录中的日志报告运行状态。初始化程序中的init_daemon函数负责生成守护进程。

流程图





Example3-7.c

/*主程序每隔10秒钟向/tmp目录中的日志3-7.log

void init_daemon(void);

int main()

{

FILE *fp;

time_t t;

init_daemon();

while(1)

{ sleep(10);

if((fp=fopen("./3-7.log","a+")) >=0)

{ t=time(0);

fprintf(fp,"守护进程还在运行，时间是：
%s",asctime(localtime(&t)));

fclose(fp);

}}}

设置守护进程

注意：fopen函数必须具有root权限。如果没有root权限，可以看到守护进程的运行，但不会在文件里写入任何字符。

chmod 设置权限

守护进程的
功能

3-7程序头文件：

#include <unistd.h>

#include <stdlib.h>

#include <signal.h>

#include <sys/param.h>

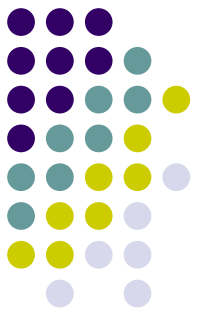
#include <sys/types.h>

#include <sys/stat.h>

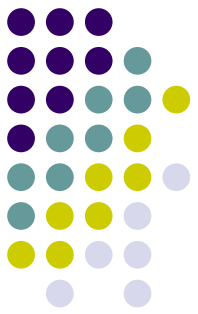
#include <stdio.h>

#include <time.h>

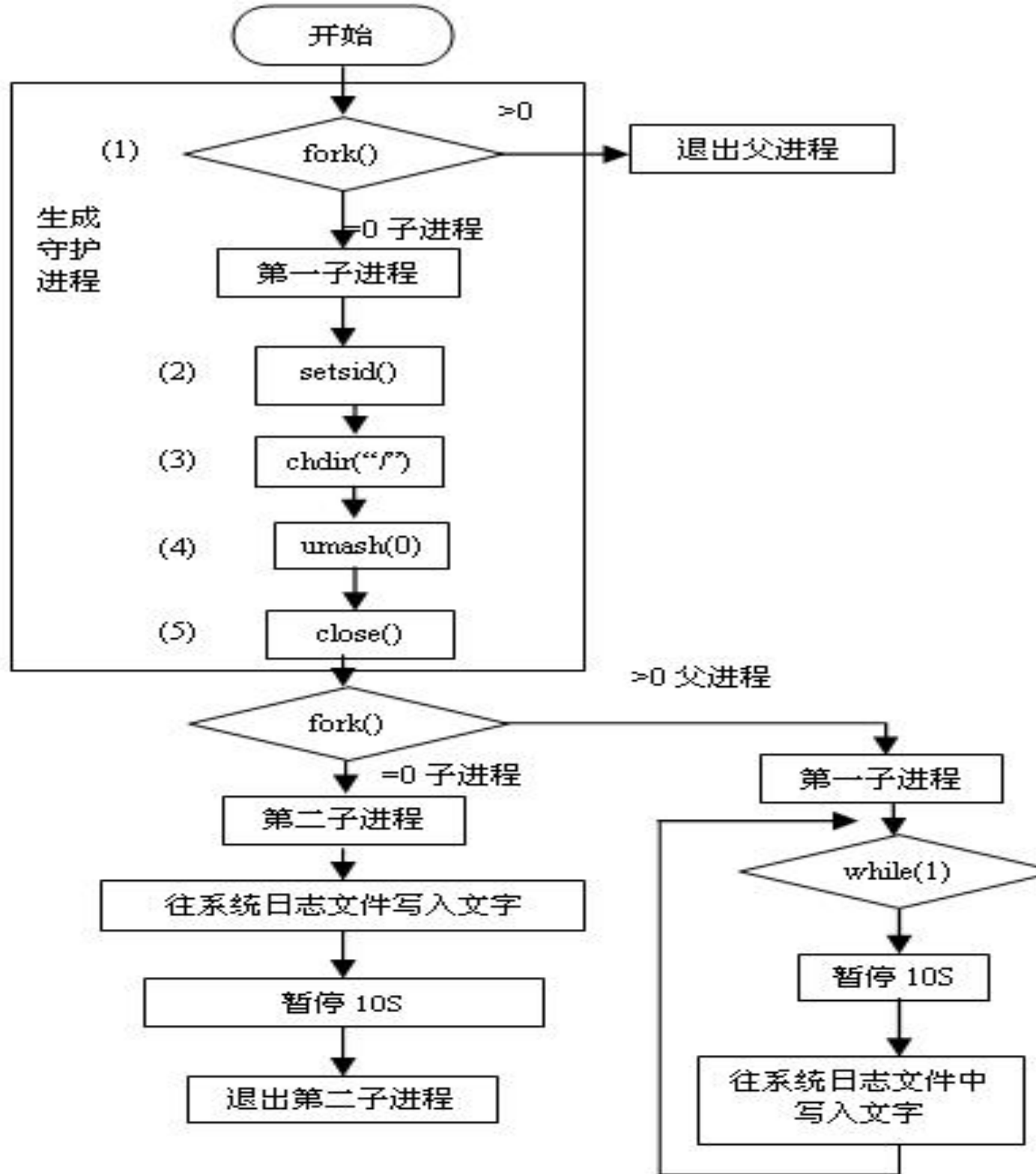
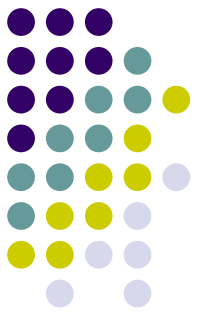
#include <stdlib.h>



```
void init_daemon(void)
{ pid_t child1,child2;  int i;
  child1=fork();
  if(child1>0)          exit(0);
  else if(child1< 0)
  { perror("创建子进程失败");
    exit(1);  }
  ①
  ② setsid();
  ③ chdir("/tmp");
  ④ umask(0);
    for(i=0;i< NOFILE;++i)
    ⑤ close(i);
    return;
  }
```



- **例3-8：** 设计一个程序，要求运行后成为守护进程，守护进程又复制出一个子进程，守护进程和它的子进程都调用**syslog**函数，把结束前的状态写入系统日志文件。




```
int main()
```

```
{
```

```
    pid_t child1,child2;
```

```
    int i;
```

```
    child1=fork();
```

```
    if(child1>0)
```

```
        exit(0);
```

```
    else if(child1< 0)
```

```
    {
```

```
        perror("创建子进程失败");/*fork失败
```

```
        exit(1);
```

```
    }
```

```
    setsid();
```

```
    chdir("/");
```

```
    umask(0);
```

```
    for(i=0;i< NOFILE;++i)
```

```
        close(i);
```

```
    openlog("例3-8程序信息",LOG_PID,LOG_DAEMON);/* 调用openlog,打  
    开日志文件*/
```

```
    child2=fork();
```

3-8程序头文件:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<sys/types.h>
```

```
#include<unistd.h>
```

```
#include<sys/wait.h>
```

```
#include<syslog.h>
```

```
#include <signal.h>
```

```
#include <sys/param.h>
```

```
#include <sys/stat.h>
```

/*(1)创建
/*这是第

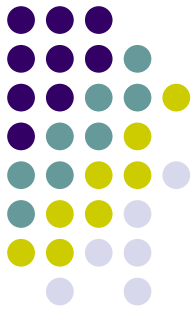
/*(2)在子进程中创建新会话*/

/*(3)改变工作目录到 "/"*/

/*(4)重设文件创建掩码*/

/*(5)关闭文件描述符*/

世界新纪元



```
if(child2==-1)
```

```
{    syslog(LOG_INFO,"创建子进程失败");
```

```
    exit(1);
```

```
}
```

```
else if(child2==0)
```

```
{    syslog(LOG_INFO,"第二子进程暂停5秒！");
```

```
    sleep(5);        /*睡眠5秒钟*/
```

```
    syslog(LOG_INFO,"第二子进程结束运行。");
```

```
    exit(0);
```

```
}
```

```
else {
```

```
    waitpid(child2,NULL,0);
```

```
    syslog(LOG_INFO,"第一子进程在等待第二子进程结束后，也  
结束运行。");
```

```
    closelog();
```

```
    /*调用closelog，关闭日志服务*/
```

```
    while(1)
```

```
{
```

```
        sleep(10);
```

```
}
```

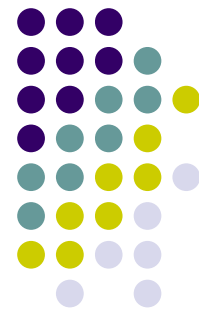
```
}
```

```
}
```

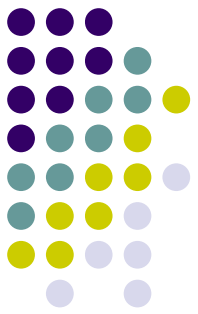
Log-INFO

Log-DEBUG

→ 打印什么级别的信息



注意：调用openlog、syslog函数，操作的系统日志文件“/var/log/syslog”或“/var/log/messages”，必须具有root权限。

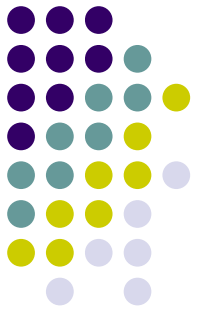


4.3.3 守护进程的编写

- openlog函数说明

- syslog函数说明

所需头文件	#include<syslog.h>
函数功能	准备做信息记录
所需头文件	#include<syslog.h>
函数功能	将信息记录至系统日志文件
函数原型	void syslog(int priority,char *format,.....);
函数传入值	priority 指定信息的种类或等级，主要有： LOG_INFO 提示相关信息 LOG_DEBUG 除错相关信息 format 参数和 printf函数相同
函数返回值	无
相关函数	openlog, closelog
备注	



掌握的知识点

- 守护进程概念
- 守护进程编写要点
- 守护进程程序应用

